Shiv Nadar University - Chennai

Department of Computer Science & Engineering

25CS2741 Fundamentals of Internet of Things + Lab

Date: 23/10/2026 (Week 12) **Continuous Lab Evaluation-12** II Yr/ 3rd Sem B.Tech CSE(IoT)A&B

Allotted Marks: 10 (Hardware Implementation + Code Efficiency + Viva-Voce)
Tools: CubeMX + Keil MDK + **STM32 Nucleo Development Board** + **Debugger**
BME280 Bosch Sensor Tech API + STM32 HAL libraries are allowed

Sensor Interfacing(BME280) – STM32(Debug) – ESP8266(Wi-Fi) HTTP – Node Analytics

BME280 Sensor Interfacing Using I²C and Bosch Sensor API

Develop an STM32-based environmental sensing application to interface a BME280 sensor using the I²C protocol and obtain compensated temperature, pressure, and humidity measurements.

Use the **Bosch Sensortec BME280 Sensor API** as the reference driver implementation. The required driver files and compensation algorithms shall be sourced from the official **BoschSensortec/BME280_SensorAPI** repository. Bosch Sensortec BME280 Sensor API

Step 1: Download and Arrange the Driver Files

Clone the official BME280_SensorAPI GitHub Repository.

You only need **three specific files** from the root folder:

- bme280.c
- bme280.h
- bme280_defs.h

Where to place them in your project directory:

1. Navigate to your Keil project directory (usually where your STM32CubeMX generated files sit).
2. Inside your Core/Src folder, copy and paste: bme280.c.
3. Inside your Core/Inc folder, copy and paste: bme280.h and bme280_defs.h

Step 2: Include the Files in Keil uVision

1. Add the .c file to the workspace:
   - Expand your target folder and locate the group named Application/User/Core
   - Right-click the group folder → Select Add Existing Files to Group...
   - Browse to Core/Src, select bme280.c, and click Add
2. **Verify Include Paths (If using standard paths):**
   - Click the **Options for Target** magic wand icon - Go to the **C/C++** tab.
   - Under **Include Paths**, ensure your project's include folder (e.g., ..\Core\Inc) is present so Keil can see bme280.h and bme280_def

---

Step 3: Implement Interface Wrappers & Read Data

Open your main.c file and construct your code using this framework:

1. Add Private Functions and Variables

```
/* USER CODE BEGIN Includes */
#include "bme280.h"
#include <stdio.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
extern I2C_HandleTypeDef hi2c1; // Assumes I2C1 is configured via CubeMX
struct bme280_dev dev;
struct bme280_data comp_data;
int8_t rslt = BME280_OK;

// Variables to store final calculated values
float temperature, pressure, humidity;
/* USER CODE END PV */
```

2. Define the Hardware Abstraction Functions

Place these functions in main.c under /* USER CODE BEGIN 4 */ at the bottom of the file:

```
/* USER CODE BEGIN 4 */
int8_t user_i2c_read(uint8_t reg_addr, uint8_t *reg_data, uint32_t len, void *intf_ptr) {
  // Dev address shifted left by 1 is handled by HAL
  uint8_t dev_addr = *(uint8_t*)intf_ptr;
  if (HAL_I2C_Mem_Read(&hi2c1, (dev_addr << 1), reg_addr, I2C_MEMADD_SIZE_8BIT, reg_data, len,
HAL_MAX_DELAY) == HAL_OK) {
    return BME280_OK;
  }
  return BME280_E_COMM_FAIL;
}

int8_t user_i2c_write(uint8_t reg_addr, const uint8_t *reg_data, uint32_t len, void *intf_ptr) {
  uint8_t dev_addr = *(uint8_t*)intf_ptr;
  if (HAL_I2C_Mem_Write(&hi2c1, (dev_addr << 1), reg_addr, I2C_MEMADD_SIZE_8BIT,
(uint8_t*)reg_data, len, HAL_MAX_DELAY) == HAL_OK) {
    return BME280_OK;
  }
  return BME280_E_COMM_FAIL;
}

void user_delay_us(uint32_t period, void *intf_ptr) {
  // HAL provides millisecond delay; convert microseconds upward
  HAL_Delay((period / 1000) + 1);
}
/* USER CODE END 4 */
```

3. Initialize and Read in main()

Inside your int main(void) function, initialize the device structure, hook up your custom communication pointers, configure the power modes, and parse the data:

```
int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_I2C1_Init();

  /* USER CODE BEGIN 2 */
  static uint8_t dev_addr = BME280_I2C_ADDR_PRIM; // 0x76 (use BME280_I2C_ADDR_SEC if SDO is high)

  // Set up the API configuration function pointers
  dev.intf_ptr = &dev_addr;
  dev.intf = BME280_I2C_INTF;
  dev.read = user_i2c_read;
  dev.write = user_i2c_write;
  dev.delay_us = user_delay_us;

  // Initialize the BME280 sensor (This automatically extracts the compensation data)
  rslt = bme280_init(&dev);

  // Configure sensor settings
  dev.settings.osr_h = BME280_OVERSAMPLING_1X;
  dev.settings.osr_p = BME280_OVERSAMPLING_1X;
  dev.settings.osr_t = BME280_OVERSAMPLING_1X;
  dev.settings.filter = BME280_FILTER_COEFF_OFF;

  rslt = bme280_set_sensor_settings(BME280_SEL_OSR_PRESS | BME280_SEL_OSR_TEMP | BME280_SEL_OSR_HUM | BME280_SEL_FILTER, &dev);

  // Set power mode to normal to enable continuous measurement cycles
  rslt = bme280_set_sensor_mode(BME280_MODE_NORMAL, &dev);
  /* USER CODE END 2 */

  /* Infinite loop */
  while (1) {
    /* USER CODE BEGIN 3 */
    // Read and automatically compensate data
    rslt = bme280_get_sensor_data(BME280_ALL, &comp_data, &dev);

    if (rslt == BME280_OK) {
      // Bosch formulas process outputs as integers or floats depending on compilation configuration
      temperature = comp_data.temperature; // °C
      pressure = comp_data.pressure / 100.0f; // Convert Pa to hPa
      humidity = comp_data.humidity; // % Relative Humidity
    }

    HAL_Delay(1000); // Poll data once per second
  }
  /* USER CODE END 3 */
```

Add private functions required to interface the Bosch API with the STM32 application.

- Configure the BME280 operating parameters such as:
  - Temperature oversampling
  - Pressure oversampling
  - Humidity oversampling
  - Sensor mode
  - Standby time
  - Filter configuration

Students should identify the appropriate structures and API functions from the Bosch driver rather than rewriting the compensation equations

Define the Hardware Abstraction Functions

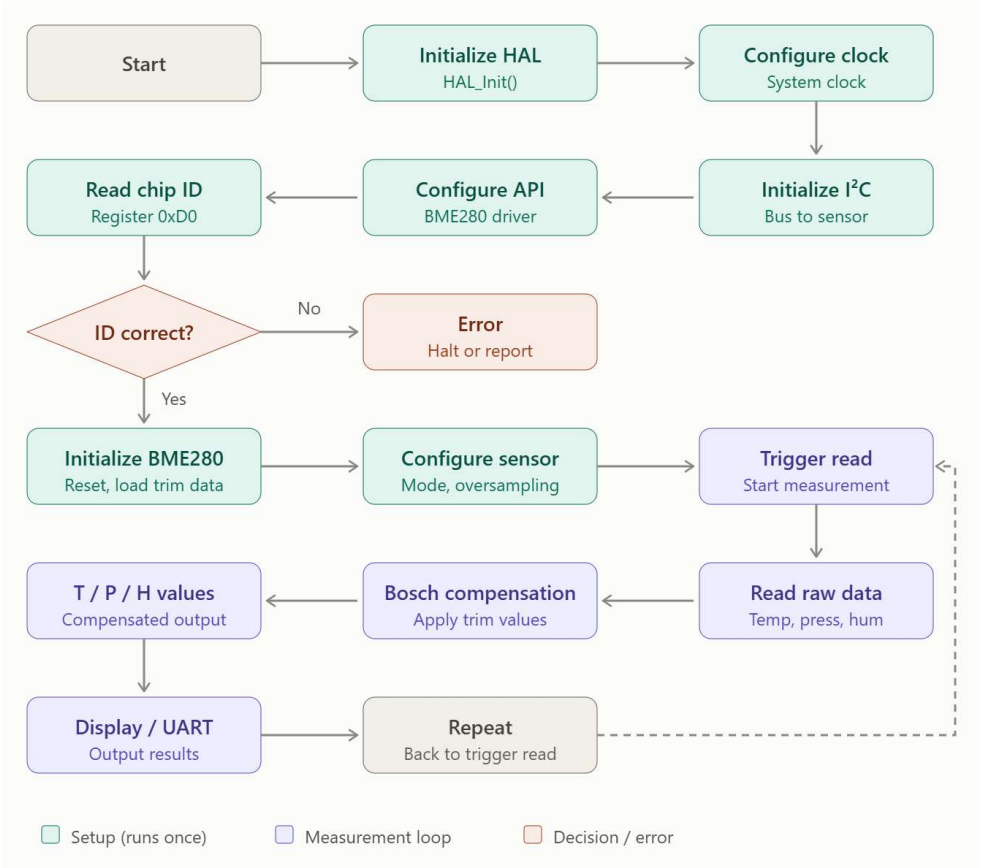
Develop the STM32-specific interface functions required by the driver.

Implement functions corresponding to:

Sensor API │ ├── Read registers │ ↓ │ STM32 I²C │ └── Write registers ↓ STM32 I²C	The implementation shall provide: read_register() write_register() delay_function()

3. Initialize and Read in main()

Develop the application flow:



Students should demonstrate:

1. **I²C communication established** between STM32 and BME280.
2. Correct **BME280 chip ID** read from the sensor.
3. Bosch API initialized successfully, Sensor configuration successfully written.
4. Raw measurement registers are successfully read.
5. Bosch compensation functions produce valid:
    - Temperature, Pressure, Humidity
6. Values are transmitted through UART.
7. Values change appropriately when the sensor environment changes.

Suggested progression

First: Read chip ID → **Second:** Read raw registers → **Third:** Use Bosch compensation → **Fourth:** Display compensated values → **Fifth:** Change oversampling/filter settings and compare the readings.

Required Output

The STM32 application shall periodically obtain and display:

Temperature : XX.XX °C

Pressure : XXXX.XX hPa

Humidity : XX.XX %RH

The output may initially be displayed through the **UART terminal**.

Once you include temperature, humidity, and pressure, the Content-Length must be calculated from the **actual JSON string length**.

Raw HTTP Response with BME280 Data

The STM32 should construct the JSON payload from the compensated BME280 readings.

For example:

{"status":"success","temperature":28.45,"humidity":62.30,"pressure":1008.72}

The complete raw HTTP response becomes:

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: <calculated_length>

{"status":"success","temperature":28.45,"humidity":62.30,"pressure":1008.72}

Expected browser response


```
{
  "status": "success",
  "temperature": 28.45,
  "humidity": 62.30,
  "pressure": 1008.72
}
```

Hint:

Construct the dynamic JSON payload

Students can use:

```
char json[200];

snprintf(json,
        sizeof(json),
        "{\"status\":\"success\","
        "\"temperature\":%.2f,"
        "\"humidity\":%.2f,"
        "\"pressure\":%.2f}",
        temperature,
        humidity,
        pressure);
```

Calculate Content-Length

```
int content_length = strlen(json);
```

Then construct the HTTP response:

```
char http_response[300];
snprintf(http_response,
        sizeof(http_response),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: application/json\r\n"
        "Content-Length: %d\r\n"
        "\r\n"
        "%s",
        content_length,
        json);
```

Send through ESP8266 using CIPSEND

```
snprintf(cmd,
        sizeof(cmd),
        "AT+CIPSEND=%d\r\n",
        response_length);
```

Next step — Node-RED

Get statistics from the BME280 data and arrive at the table as below

Parameter	Minimum	Maximum	Average
Temperature (°C)	27.91	29.02	28.46
Humidity (%RH)	59.80	64.20	62.15
Pressure (hPa)	1007.84	1009.31	1008.56